

MEDIATEK

MT8676 Android OTA User Manual

版本: 1.0
出版日期: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-08-12	Ming Ji	正式版

目录

版本记录	2
目录	3
图片目录	4
表格目录	4
1 OTA	5
1.1 概述	5
1.1.1 目的	5
1.1.2 目标读者	5
1.1.3 如何使用本文档	5
1.2 参考文件	6
1.3 OTA 升级的定义	6
1.4 OTA 缩略词	7
1.5 打开 A/B 系统升级	7
1.6 架构概述	7
1.6.1 Preloader 启动控制流程	8
1.6.2 A/B 系统分区布局	8
1.6.3 A/B 系统升级中 Ramdisk 的存放位置	9
1.6.4 Boot Control HAL and HIDL	10
1.6.5 A/B 系统升级进程	12
1.6.6 虚拟 A/B	12
1.7 生成 A/B 系统 OTA 包的指南	13
1.7.1 利用 split build 1.0 (LD1.0) 逐步生成 A/B 系统 OTA 升级包	13
1.7.2 利用 split build 2.0 (LD2.0) 逐步生成 A/B 系统 OTA 升级包	16
1.8 A/B 系统升级的限制	18
1.9 如何使用和调试 A/B 系统升级	18
1.9.1 如何使用 A/B 系统升级	18
1.9.2 如何调试 A/B 系统升级	19
1.10 常见问题	19
1.10.1 设备是否需要同时烧录 A/B 分区?	19
1.10.2 如何确定 AB 系统当前运行在哪个分区 (A 或 B) ?	19

1.10.3 当系统从 slot_a 启动时，是否允许去加载 B 分区（例如：系统从 A 分区启动，LK 阶段去加载 boot_b 分区）？ 20

1.10.4 需要 AB 升级的分区需要满足什么条件？ 20

1.10.5 如何去打开和关闭 A/B 系统？ 20

1.10.6 A/B 与 non-A/B 之间是否允许相互升级？ 20

1.10.7 系统如何去设定 boot_sucessfully 标志位？ 20

附件一 附加条款.....21

图片目录

图 1-1. Preloader 启动控制流程.....8

表格目录

表 1-1. 章节概述.....6

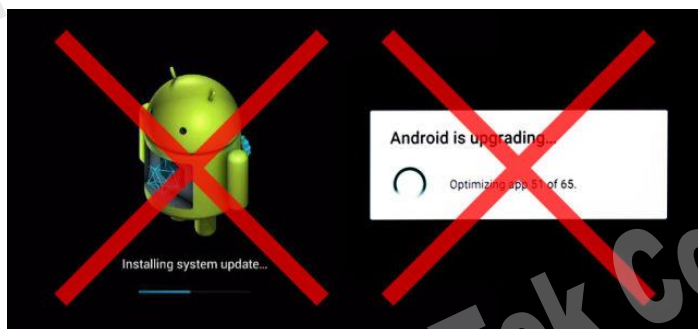
表 1-2. 缩略词7

1 OTA

1.1 概述

A/B 系统更新的介绍如下:

- 这是一项新的 OTA 功能，又称为无缝更新。
- 设备不会变砖（从新系统启动失败后可以回滚到旧系统）。
- 磁盘上始终保留一个可启动的系统。
- OTA 升级保持在正常模式的后台运行，用户无感。



1.1.1 目的

本文档为用户提供了 A/B 系统更新的指南。它描述了如何在 Android 平台上生成 OTA 包。本手册还详细说明了在联发科技平台上启用 A/B 系统更新（无缝更新）功能所需的机制。

1.1.2 目标读者

本文档主要面向如下群体：

- 具备 OTA A/B 系统更新（无缝更新）技术知识的工程师

1.1.3 如何使用本文档

本部分解释了本文档中信息的分布方式，并提供了一些提示和示例，以简化在本文档中查找和理解信息。下表概述了本文档中的章节和附录。

表 1-1. 章节概述

#	章节	描述
1	概述	描述了本文档的范围和布局
2	参考文件	参考网站或文档
3	OTA 升级的定义	规定了 OTA 升级的定义
4	OTA 缩略词	列举了 OTA 缩略语
5	打开 A/B 系统升级	在联发科技平台打开 AB 系统升级
6	架构概述	AB 系统升级的架构介绍
7	生成 A/B 系统 OTA 包的指南	介绍如何制作 OTA 全量升级包和差分升级包
8	A/B 系统升级的限制	列出了 A/B 系统升级的限制
9	如何使用和调试 A/B 升级	使用和调试 A/B 升级的方法
10	常见问题	给出了常见问题及答案

1.2 参考文件

- [1] The Android OTA Package Tools: <https://source.android.com/devices/tech/ota/tools.html>
- [2] A/B (Seamless) System updates:
https://source.android.com/devices/tech/ota/ab_updates
https://source.android.com/devices/tech/ota/inside_packages.html
- [3] Implementing A/B Updates: https://source.android.com/devices/tech/ota/ab_implement
- [4] Virtual A/B Updates: https://source.android.com/docs/core/ota/virtual_ab
- [5] Configure ART: <https://source.android.com/devices/tech/dalvik/configure>
- [6] Ramdisk storage location: <https://source.android.com/docs/core/architecture/partitions/generic-boot>

1.3 OTA 升级的定义

OTA 的专业术语定义如下：

全包升级： 全包升级是指对设备的待升级分区数据进行完整的更新，升级包内包含各个分区的完整 image 信息。

增量升级： 增量升级是指对设备的待升级分区数据与目标版本对应分区数据的差异部分进行更新，升级包内只包含两个版本的差异部分，这能极大缩减升级包的大小。

批注： 只能在编译差分包时使用的旧版本或源版本的设备上安装对应的增量更新包。

虚拟 A/B： Google 在 A/B 的基础上新增了虚拟 A/B 功能，可以使用由设备映射器创建的 COW 设备来节省超级分区大小。

1.4 OTA 缩略词

本文档使用的缩略语如下：

表 1-2. 缩略词

缩略词	全称	释义
LK	Little Kernel	微型内核
OTA	Over-The-Air	空中下载升级

1.5 打开 A/B 系统升级

请在以下文件中将 **MTK_AB_OTA_UPDATER** 设置为 **yes**（如果相应文件中不存在 **MTK_AB_OTA_UPDATER**，请在文件中添加此配置）：

- **ProjectConfig.mk:**
 - device/mediatek/projects/<project_name>/ProjectConfig.mk
- **Preloader:**
 - vendor/mediatek/proprietary/bootable/bootloader/preloader/custom/<project_name>/<project_name>.mk
- **SystemConfig.mk:**
 - device/mediatek/system/<mssi_xxxx>/SystemConfig.mk

如果项目使用 **LK**：

- **LK:**
 - vendor/mediatek/proprietary/bootable/bootloader/lk/project/<project_name>.mk

如果项目使用 **LK2**：

- **LK2:**
 - vendor/mediatek/proprietary/bootable/bootloader/lk2/project/<project_name>.mk

1.6 架构概述

本章节简要描述系统的各个模块及其关系。

1.6.1 Preloader 启动控制流程

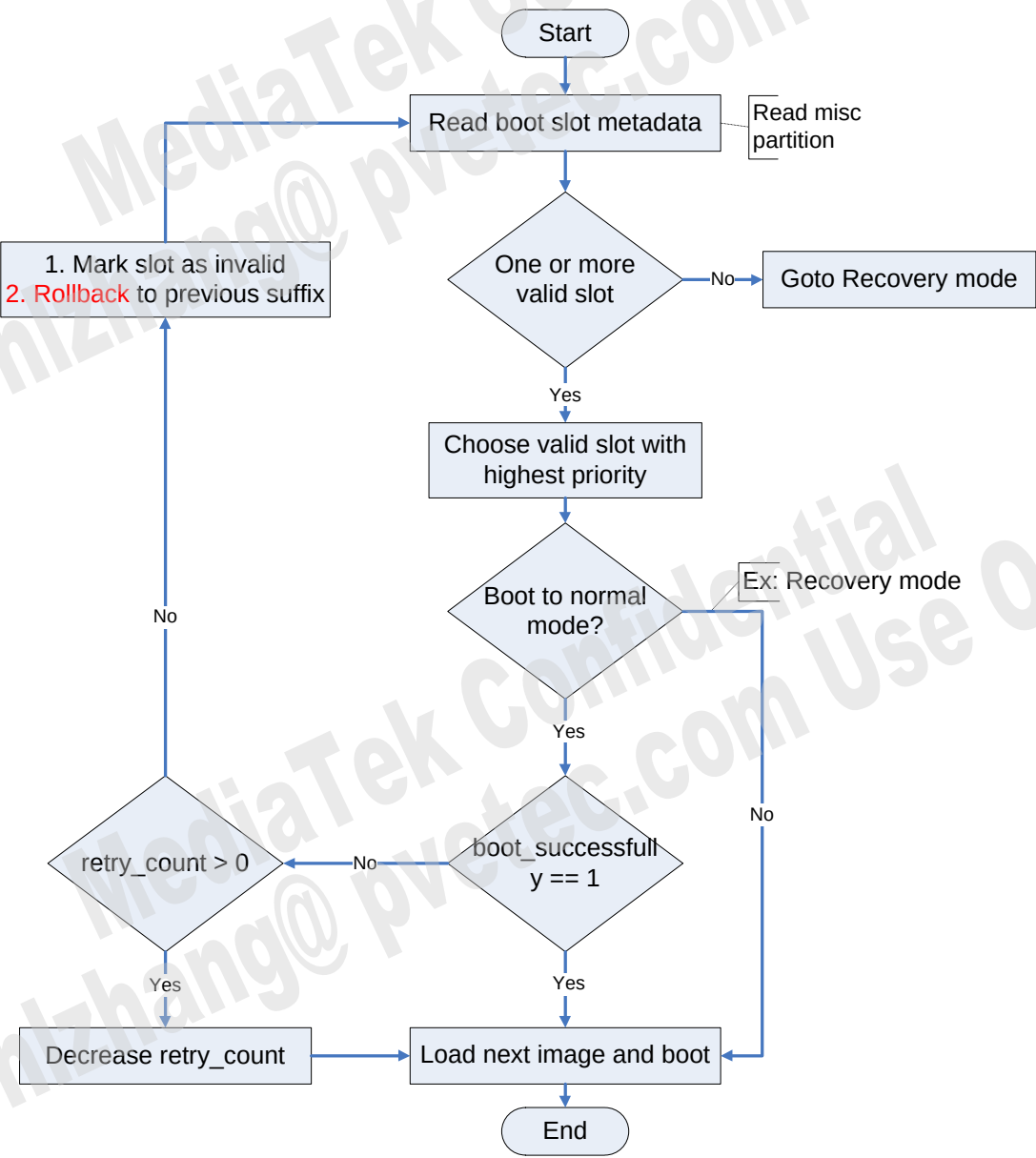
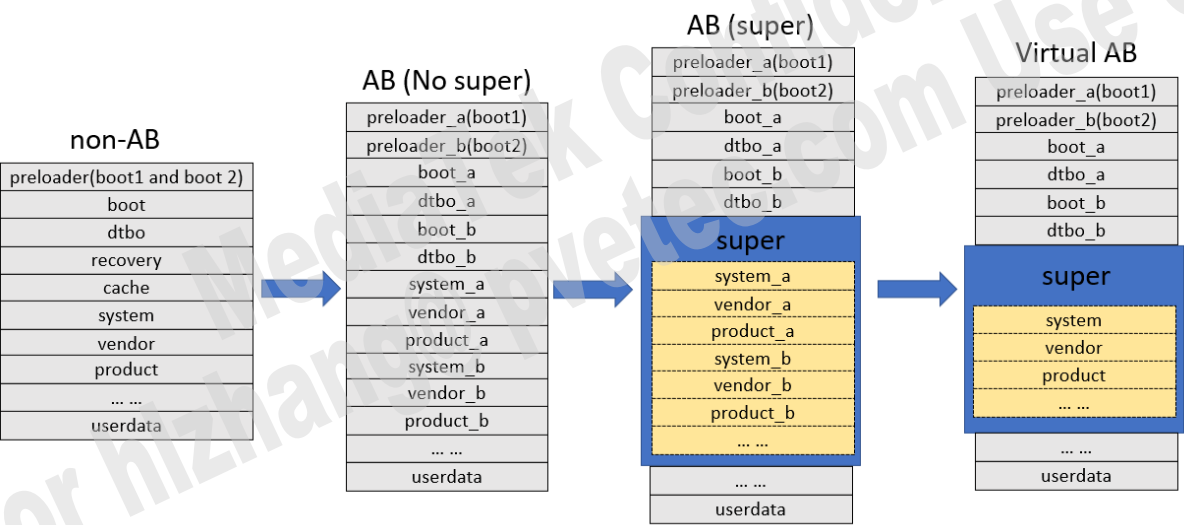


图 1-1. Preloader 启动控制流程

1.6.2 A/B 系统分区布局

各种类型系统的分区布局如下：



各种系统的分区布局差异如下：

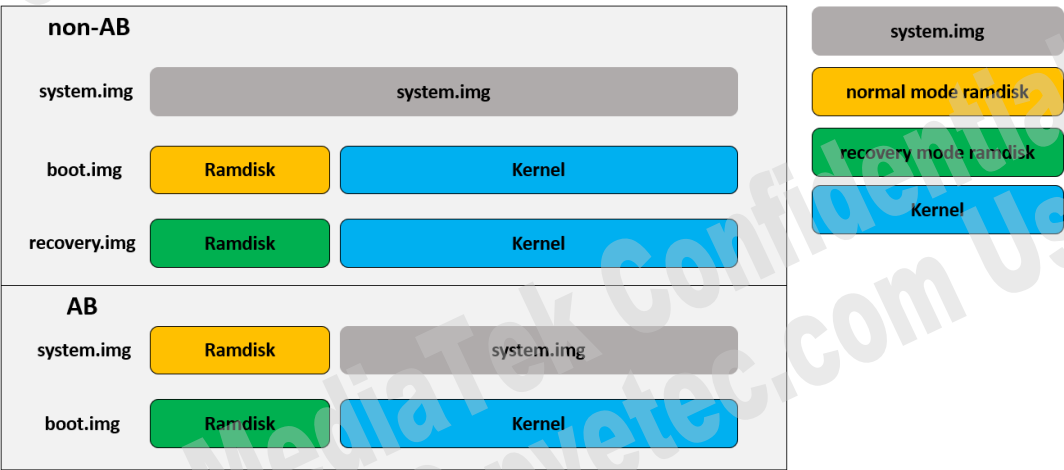
- AB 系统不再需要 recovery 和 cache 分区。
- 从 Android Q 开始，Google 增加了一个动态分区功能，将 system、vendor、product 等分区统一放在 super 分区内。
- 从 Android R 开始，Google 增加了虚拟 A/B 功能。系统只会保存一份当前系统的 system、vendor、product 等 super 子分区的镜像信息，从而节省 super 分区占用的存储空间。

1.6.3 A/B 系统升级中 Ramdisk 的存放位置

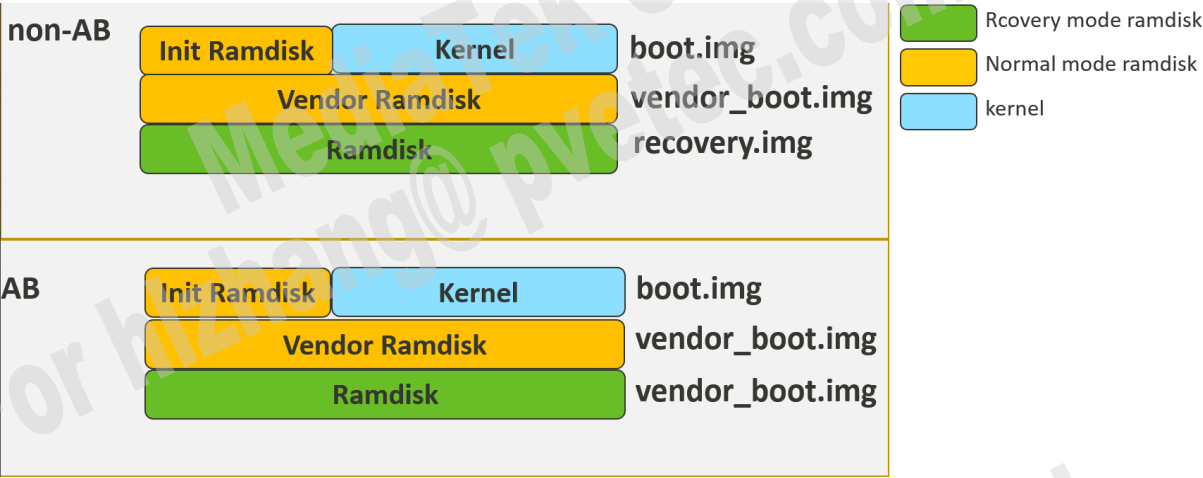
不同 Android 版本内 ramdisk 信息的存储位置存在差异，详细信息请参考 Google 文档说明：

<https://source.android.com/docs/core/architecture/partitions/generic-boot>。

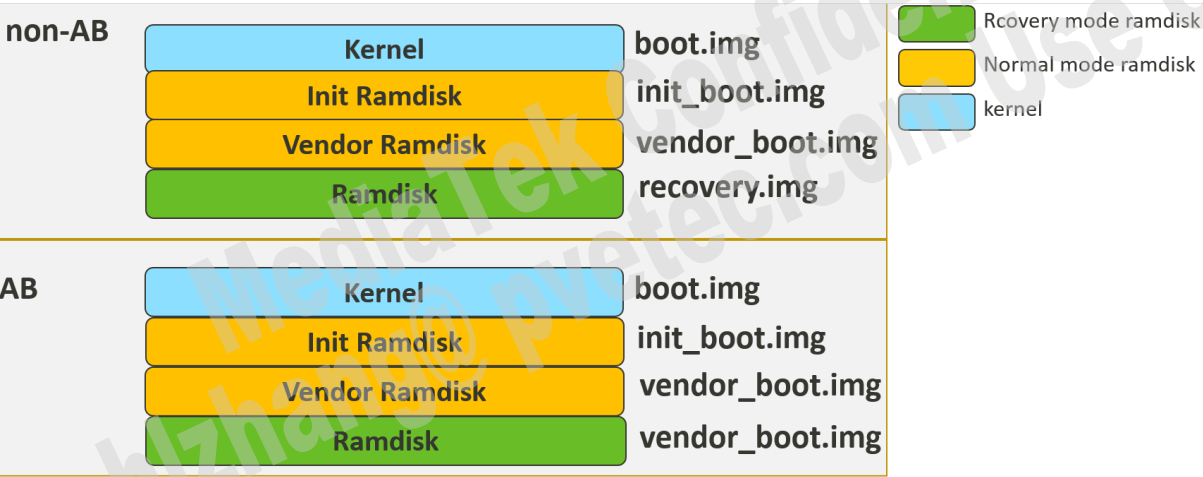
1. 如果系统的 Android 版本低于 Android R，其 ramdisk 的打包位置如下：



2. 如果系统的 Android 版本是 Android R 或 Android S，其 ramdisk 的打包位置如下：



3. 从 Android T 开始，其 ramdisk 的打包位置如下：



1.6.4 Boot Control HAL and HIDL

联发科技使用 AOSP 默认的 boot control HIDL 接口来访问 boot control HAL。

bootctl 的源代码路径：`hardware/interfaces/boot`

Boot control HAL for update_engine

- Android version $\geq$ U:
 - bootctl 的源代码路径：`vendor/mediatek/proprietary/hardware/bootctrl_service/`
- Android version $\geq$ R:
 - bootctl 的源代码路径：`vendor/mediatek/proprietary/hardware/bootctrl/`

Preloader (PL)和 LK 也有 boot control API。

- **PL:**
 - vendor/mediatek/proprietary/bootable/bootloader/preloader/platform/common/bootctrl/
- **LK:**
 - vendor/mediatek/proprietary/bootable/bootloader/lk/platform/common/bootctrl/1.0

当设备首次启动时，preloader 阶段会去初始化 bootctl 参数，将其设定为默认初始值。

Default value in misc partition	
slot A	slot B
priority = 15 tries_remaining= 7 successful_boot = 1	priority = 15 tries_remaining = 7 successful_boot = 0

在设备首次完成 OTA 升级后，它将从 slot_b 启动，因为 slot_b 比 slot_a 的优先级更高。

Default value in misc partition	
slot A	slot B
priority = 14 tries_remaining = 1 successful_boot = 1	priority = 15 tries_remaining = 7 successful_boot = 0

设备首次成功从 slot_b 启动后，slot_b 的 successful_boot 将被设置为 1。

Default value in misc partition	
slot A	slot B
priority = 14 tries_remaining = 1 successful_boot = 1	priority = 15 tries_remaining = 5 successful_boot = 1

当设备第二次从 slot_b 成功启动后，slot_b 的 tries_remaining 会被设置为 1。

Default value in misc partition	
slot A	slot B
priority = 14 tries_remaining = 1 successful_boot = 1	priority = 15 tries_remaining = 1 successful_boot = 1

1.6.5 A/B 系统升级进程

- **Update_engine:**
 - 由 update_engine.rc 执行
 - 仅用于更新分区信息
 - 使用 update.zip 中的 payload.bin 来更新每个分区
 - 在 CPU 小核和系统后台运行
 - 调用 boot_control HAL/HIDL 接口
 - 在升级完所有的分区后，系统将从新分区执行一个 postinstall 程序

Update_engine 无法实现如下功能：

- 升级分区表 (修改分区布局)
- 修改当前 slot 分区内的内容

- **Update_engine_client:**
 - Update_engine_client 仅用于调试，此进程可以获取更新状态
- **Update_verifier:**
 - Update_verifier 由 update_verifier.rc 执行
 - 如果产品支持 verity，检查 verity 模式是否为强制模式
 - 平台成功启动后将 slot 的 boot successful flag 置 1

1.6.6 虚拟 A/B

- 虚拟 A/B 在 Android R 中引入，可以使用 COW 设备来节省超级分区大小，但在更新流程完成后，还需要在新系统重启后将 COW 设备内的数据合并到 super 分区内组成新的 super.img。
- 在搭载 Android 11 及更高版本的设备上，虚拟 A/B 是 GMS 要求。

- 限制:
 - 虚拟 A/B 不支持在 **recovery** 模式下进行差分升级，Google 提供的虚拟 A/B 文档内对于该限制的描述如下：

The userdata partition cannot be mounted under recovery if it is encrypted. Free Blocks within the userdata partition cannot be determined, so we cannot allocate COW files on the userdata partition. Hence, we cannot create new snapshots in recovery. Sideload an incremental OTA on a Virtual A/B device in recovery is impossible and unsupported
- 虚拟 A/B 这个功能默认是开启状态，但是它可以通过修改编译流程来关闭。
- 在 `build/make/target/product/virtual_ab_ota.mk` 中，`PRODUCT_VIRTUAL_AB_OTA` 及其属性已分配，因此可以通过跳过此 `makefile` 并使用传统的 A/B 升级来禁用虚拟 A/B OTA，具体方法是删除 `device/mediatek/system/common/device.mk` 和 `device/mediatek/vendor/common/device.mk` 中的以下行，然后重新执行 `split build`。

```
$(call inherit-product, $(SRC_TARGET_DIR)/product/virtual_ab_ota.mk)
```

1.7 生成 A/B 系统 OTA 包的指南

本章节描述了一组生成 A/B 系统 OTA 包的指南。从 Android Q 开始，联发科技默认仅支持 `split build`。请根据项目使用的 `split build` 类型选择相应的编译方式。

1.7.1 利用 `split build 1.0 (LD1.0)` 逐步生成 A/B 系统 OTA 升级包

1.7.1.1 Full Build Project with Full `otapackage.zip` and `targefiles.zip`

1. Get `split build 1.0 (LD1.0)` command

执行以下命令以获取完整的 `split build` 命令：

```
./vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py full_${PROJ_NAME}-  
${eng|userdebug|user}
```

例如：

- 输入指令：

```
./vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py full_tb8786p1_64-  
userdebug
```

- 输出指令：

```
source build/envsetup.sh && export OUT_DIR=out_sys && lunch sys_mssi_t_64_cn-userdebug &&  
make sys_images
```

```
source build/envsetup.sh && export OUT_DIR=out && lunch vnd_tb8786p1_64-userdebug && make vnd_images krn_images
```

```
python out_sys/target/product/mssi_t_64_cn/images/split_build.py --system-dir out_sys/target/product/mssi_t_64_cn/images --vendor-dir out/target/product/tb8786p1_64/images --kernel-dir out/target/product/tb8786p1_64/images --output-dir out/target/product/tb8786p1_64/merged
```

以下三条指令分别代表：

- **Build system**

```
source build/envsetup.sh && export OUT_DIR=out_sys && lunch sys_mssi_t_64_cn-userdebug && make sys_images
```

- **Build vendor and kernel**

```
source build/envsetup.sh && export OUT_DIR=out && lunch vnd_tb8786p1_64-userdebug && make vnd_images krn_image
```

- **Merge**

```
python out_sys/target/product/mssi_t_64_cn/images/split_build.py --system-dir out_sys/target/product/mssi_t_64_cn/images --vendor-dir out/target/product/tb8786p1_64/images --kernel-dir out/target/product/tb8786p1_64/images --output-dir out/target/product/tb8786p1_64/merged
```

2. Build full otapackage.zip

如果您需要编译全量升级包，请在 merge 指令后面添加 '**--otapackage**' 参数。

例如：

```
python out_sys/target/product/mssi_t_64_cn/images/split_build.py --system-dir out_sys/target/product/mssi_t_64_cn/images --vendor-dir out/target/product/tb8786p1_64/images --kernel-dir out/target/product/tb8786p1_64/images --output-dir out/target/product/tb8786p1_64/merged --otapackage
```

全包输出路径：

例如：out/target/product/\${PROJ}/merged/otapackage.zip

3. Build targetfiles.zip

如果您需要编译 targetfiles.zip，请在 merge 指令后面添加 '**--targetfiles**' 参数。

例如：

```
python out_sys/target/product/mssi_t_64_cn/images/split_build.py --system-dir out_sys/target/product/mssi_t_64_cn/images --vendor-dir out/target/product/tb8786p1_64/images
```

```
--kernel-dir out/target/product/tb8786p1_64/images
--output-dir out/target/product/tb8786p1_64/merged
--targetfiles
```

Target 包输出路径:

例如: out/target/product/\${PROJ}/merged/ target_files.zip

1.7.1.2 Build Incremental Package

首先将当前版本定义为 **Source** 版本, 需要升级上去的版本定义为 **Target** 版本。如下是编译差分包的步骤:

1. 请按照小节 1.7.1.1 Full Build Project with Full otapackage.zip and targetfiles.zip 内的说明执行 split build, 并获取 Source 版本和 Target 版本的 targetfiles.zip。请务必确保 Target 版本的编译时间要新于 Source 版本的编译时间。
2. 将 Source 版本的 targetfiles.zip 重命名为 "source.zip", 将 Target 版本的 targetfiles.zip 重命名为 "target.zip"。
3. 在项目编译环境中, 依次执行以下命令:

Source & Lunch:

```
source build/envsetup.sh && export OUT_DIR=out_sys && lunch sys_${SYS_RPOJ}-
${eng|userdebug|user}
```

```
source build/envsetup.sh && export OUT_DIR=out && lunch vnd_${PROJ}-${SYS_PROJ};
```

Centralize tools to out/host/linux-x86:

```
cp out_sys/host/linux-x86/* out/host/linux-x86/ -rf
```

```
cp out_sys/soong/host/linux-x86/* out/host/linux-x86/ -rf
```

```
cp out/soong/host/linux-x86/* out/host/linux-x86/ -rf
```

Build incremental package:

- **Android version ≤ Android T:**

```
python3 build/tools/releasetools/ota_from_target_files --block -v -p out/host/linux-x86 -k
device/mediatek/common/security/releasekey -i source.zip target.zip delta.zip
```

- **Android version ≥ Android U:**

```
./out/host/linux-x86/bin/ota_from_target_files -v -p out/host/linux-x86 -k
device/mediatek/common/security/releasekey -i source.zip target.zip delta.zip
```

如果您的项目没有 **releasekey**, 请删除指令内的 "-k device/mediatek/common/security/releasekey", 系统将使用默认密钥 (test key)。或者使用 -k 指定您项目使用的密钥路径。

- **source.zip:** Source 版本的 targetfiles.zip。
- **target.zip:** Target 版本的 targetfiles.zip。

- **delta.zip**: 从 Source 版本升级到 Target 版本的差分升级包。

批注: targetfiles.zip 和差分升级包的命名没有具体限制, 客户可以根据自己的喜好设置名称。

1.7.2 利用 split build 2.0 (LD2.0) 逐步生成 A/B 系统 OTA 升级包

1.7.2.1 Full Build Project with Full otapackage.zip and targetfiles.zip

1. Get split build 2.0 (LD2.0) command

执行以下命令以获取完整的 split build 命令:

```
./vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py full_${PROJ_NAME}-  
${eng|userdebug|user}
```

例如:

- 输入指令:

```
./vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py full_  
auto8676p1_64_bsp-userdebug
```

- 输出指令:

```
source build/envsetup.sh && export OUT_DIR=out_sys && lunch sys_mssi_auto_64_cn_armv82_car-  
userdebug && make sys_images
```

```
source build/envsetup.sh && export OUT_DIR=out_hal && lunch hal_mgvi_spm_64_armv82-userdebug  
&& make hal_images
```

```
source build/envsetup.sh && export OUT_DIR=out_krn && lunch krn_mgk_64_k61-userdebug && make  
krn_images
```

```
source build/envsetup.sh && export OUT_DIR=out && lunch vext_auto8676p1_64_bsp-userdebug &&  
make vext_images
```

```
python out_sys/target/product/mssi_spm_64_cn_armv82/images/split_build.py  
--system-dir out_sys/target/product/mssi_spm_64_cn_armv82/images --vendor-dir  
out_hal/target/product/mgvi_spm_64_armv82/images  
--kernel-dir out_krn/target/product/mgk_64_k61/images  
--vext-dir out/target/product/auto8676p1_64_bsp/images  
--output-dir out/target/product/auto8676p1_64_bsp/merged
```

这五条指令分别代表:

- **Build system**

```
source build/envsetup.sh && export OUT_DIR=out_sys && lunch sys_mssi_auto_64_cn_armv82_car-  
userdebug && make sys_images
```

- **Build hal**

```
source build/envsetup.sh && export OUT_DIR=out_hal && lunch hal_mgvi_spm_64_armv82-userdebug && make hal_images
```

- **Build kernel**

```
source build/envsetup.sh && export OUT_DIR=out_krn && lunch krn_mgk_64_k61-userdebug && make krn_images
```

- **Build vext**

```
source build/envsetup.sh && export OUT_DIR=out && lunch vext_auto8676p1_64_bsp-userdebug && make vext_images
```

- **Merge**

```
python out_sys/target/product/mssi_t_64_cn/images/split_build.py  
--system-dir out_sys/target/product/mssi_t_64_cn/images  
--vendor-dir out/target/product/tb8786p1_64/images
```

2. Build full otapackage.zip

如果您需要编译全量升级包，请在 merge 指令后面添加 '--otapackage' 参数。

例如：

```
python out_sys/target/product/mssi_spm_64_cn_armv82/images/split_build.py  
--system-dir out_sys/target/product/mssi_spm_64_cn_armv82/images  
--vendor-dir out_hal/target/product/mgvi_spm_64_armv82/images  
--kernel-dir out_krn/target/product/mgk_64_k61/images  
--vext-dir out/target/product/auto8676p1_64_bsp/images  
--output-dir out/target/product/auto8676p1_64_bsp/merged  
--otapackage
```

全量升级包路径：

例如：out/target/product/\${PROJ}/merged/otapackage.zip

3. Build targetfiles.zip

如果您需要编译 targetfiles.zip，请在 merge 指令后面添加 '--targetfiles' 参数。

例如：

```
python out_sys/target/product/mssi_spm_64_cn_armv82/images/split_build.py  
--system-dir out_sys/target/product/mssi_spm_64_cn_armv82/images  
--vendor-dir out_hal/target/product/mgvi_spm_64_armv82/images  
--kernel-dir out_krn/target/product/mgk_64_k61/images  
--vext-dir out/target/product/auto8676p1_64_bsp/images  
--output-dir out/target/product/auto8676p1_64_bsp/merged  
--targetfiles
```

Target 包输出路径：

例如: out/target/product/\${PROJ}/merged/ target_files.zip

1.7.2.2 Build Incremental Package

请参阅小节 1.7.1.2 Build Incremental Package 中的步骤来编译差分升级包。

1.8 A/B 系统升级的限制

本章节描述了 A/B 系统更新的限制。

- OTA 不支持修改分区表和更改分区布局
- 不支持修改当前插槽的分区内容
- 不支持 non-AB 和 AB 之间的相互升级

1.9 如何使用和调试 A/B 系统升级

1.9.1 如何使用 A/B 系统升级

- **Normal mode:**
 - PC must install python
 - Put update_device.py (system/update_engine/scripts/update_device.py) to adb folder
 - Connect phone with PC via usb cable
 - python update_device.py --file update.zip in adb window

```
D:\Adb>python update_device.py full_k53v1_64_ab-ota-1502713905.zip
INFO:root:Running: adb reverse tcp:1234 tcp:51501
INFO:root:Running: adb shell su 0 update_engine_client --update --follow
HASH=KyKz6JU+K3FQwYimMurD13xU0WG02n3bESPb0t02sLM=
FILE_SIZE=592100123
METADATA_HASH=ytU6gL3hIdEzi9rnkWEeu1UXnTK1HSxTzAoA/fAEzD0=
METADATA_SIZE=86389
USER_AGENT=Dalvik <something, something>
NETWORK_ID=0
```

- **Recovery mode:**
 - ADB sideload update (**VAB only supports full OTA**):
 - adb reboot recovery
 - choose “Apply update form ADB” on recovery UI
 - adb sideload d:\update.zip
 - SD card update:
 - adb push update.zip /storage/F205-EFBE (/storage/F205-EFBE is SD card mount directory)
 - adb reboot recovery

- choose “Apply update form SD card” on recovery UI
- go on choosing the update.zip fi

1.9.2 如何调试 A/B 系统升级

Log location

- **Normal mode:**
 - 方法一：
 - mtklog\mobilelog\APLogxxx\main_log
 - Search **update_engine** key word
 - 方法二：
 - adb pull data/misc/update_engine_log
 - Search **update_engine** key word
- **Recovery mode:**
 - Adb pull /tmp/recovery.log

Common error code

- 所有 error code 的介绍都在 system/update_engine/common/error_code.h

1.10 常见问题

1.10.1 设备是否需要同时烧录 A/B 分区？

- 不需要，只有 preloader 分区需要同时烧录 preloader_a 与 preloader_b 分区，其它的分区都默认只需要烧录 A 分区。
- 在 flashtool 烧录完成后，平台默认从 A 分区启动。
- 当 B 分区为空时，A 分区仍然可以通过 OTA 差分升级到 B 分区。

1.10.2 如何确定 AB 系统当前运行在哪个分区（A 或 B）？

- 检查属性 ro.boot.slot_suffix (getprop ro.boot.slot_suffix)
- 在命令行中检查 androidboot.slot_suffix (/proc/cmdline)

1.10.3 当系统从 **slot_a** 启动时，是否允许去加载 **B** 分区（例如：系统从 **A** 分区启动，**LK** 阶段去加载 **boot_b** 分区）？

- 不可以，系统启动 slot 和分区后缀必须保持一致，即 slot_a 启动只能加载 A 分区，slot_b 启动只能加载 B 分区。

1.10.4 需要 **AB** 升级的分区需要满足什么条件？

- 如果分区需要 OTA 更新，该分区必须同时具有 _a 和 _b 分区（super 分区除外）。

1.10.5 如何去打开和关闭 **A/B** 系统？

- 通过设定 MTK_AB_OTA_UPDATER 的，打开 A/B 设置为 yes，关闭 A/B 设置为 no。

1.10.6 **A/B** 与 **non-A/B** 之间是否允许相互升级？

- 不行，A/B 与 non-A/B 不允许相互升级。

1.10.7 系统如何去设定 **boot_sucessfully** 标志位？

- Update_verfier 进程会去设置 boot_sucessfully
- 源代码位于 bootable/recovery/update_verifier/update_verifier.cpp

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。